Department of Electrical and Information Engineering

University of Ruhuna

Project Analysis Report
High Performance Computing
EE7218 / EC7207

Performance Analysis of Parallel Image Convolution

using OpenMP, POSIX Threads, MPI and CUDA

Group Number: 40

EG/2021/4735	Ranasingha K.K.M.P.
EG/2021/4736	Ranasinghe E.W.G.I.S.
EG/2021/4738	Ranasinghe R.M.W.O.

May 24, 2026

Contents

1 Introduction

Image convolution is a crucial process for computer vision and image processing applications. Given an input image I and a convolution kernel K of size $k \times k$, the output pixel value at position (x, y) and colour channel c is:

$$O(x,y,c) \;=\; \sum_{ky=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} \sum_{kx=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} I(\text{clamp}(x+kx, 0, W-1),\, \text{clamp}(y+ky, 0, H-1),\, c) \cdot K(kx, ky) \tag{1}$$

For a $W \times H$ image with C channels, Eq. (1) requires $W \cdot H \cdot C \cdot k^2$ multiply-accumulate operations. For our Gaussian blur kernel ($k = 21$, $k^2 = 441$) applied on a 2000×1333 image with $C = 3$ channels this results in $\approx 3.5 \times 10^9$ operations, hence clearly motivating parallelism. We also evaluate a 3×3 edge detection kernel and a 3×3 sharpen kernel to contrast compute-bound vs. memory-bound behaviour.

This report presents five parallel implementations (OpenMP, POSIX Pthreads, MPI, CUDA, and a Hybrid MPI+OpenMP approach) with an analysis based on correctness compared to the serial baseline using Root Mean Square Error (RMSE), and benchmarks execution time as a function of the degree of parallelism.

2 Parallel Programming Concepts Applied

2.1 Overview of the Parallelisation Strategy

Parallel implementations leverage the inherently parallelizable nature of 2D convolution, every output pixel calculation occurs independently based on its local neighborhood of read-only input pixels. No communication between the pixels is necessary in the computation. Load balancing and result assembly are the only places where coordination is necessary.

The primary decomposition technique used is row-block decomposition, which involves dividing the rows into continuous segments to be processed by the parallel workers (threads or processes), ensuring each worker accesses a contiguous memory region.

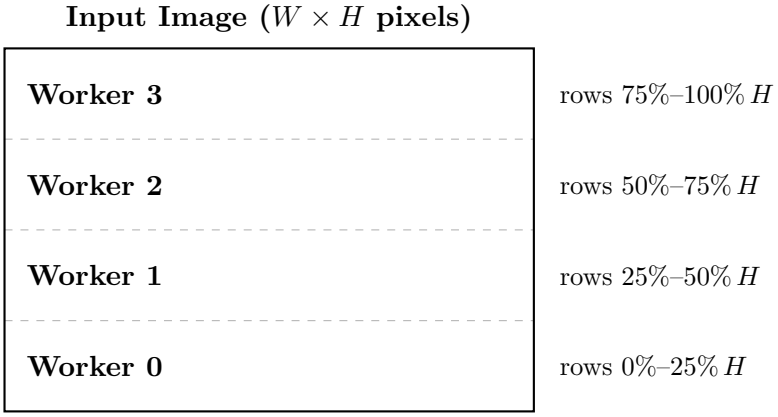

Figure 1: Row-block partitioning strategy. The image is divided into contiguous horizontal strips and each strip is assigned to one parallel worker (thread or process).

2.2 Serial Baseline

The serial implementation iterates over every (y, x, c) triplet in a triple nested loop and call apply_kernel function for each pixel. Boundary pixels are handled by clamping coordinates to $[0, W-1]$ and $[0, H-1]$.

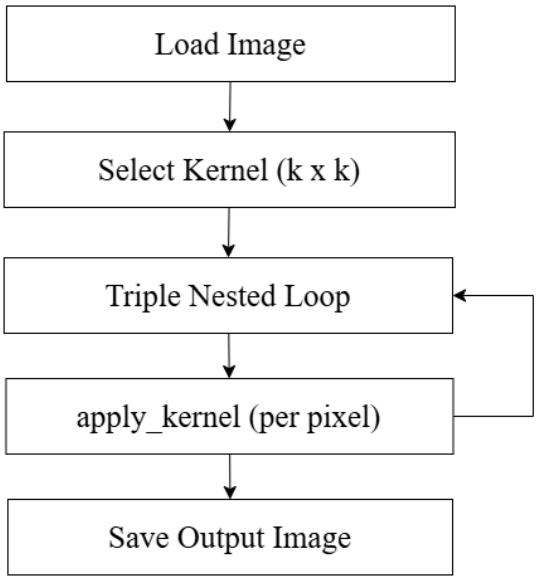

Figure 2: Serial convolution control flow

2.3 OpenMP — Shared-Memory Thread Parallelism

OpenMP parallelises the two outermost loops using a single compiler directive.

Key concepts:

- `collapse(2)` combines the two y-x loops into a one iteration space of dimension $H \times W$, allowing for more detailed scheduling.
- `schedule(dynamic)` assigns chunks of the combined iteration space to idle threads during execution, resulting in better load balancing.
- All threads access same read-only input and kernel; each thread writes to unique output locations. Therefore, no mutex is required.

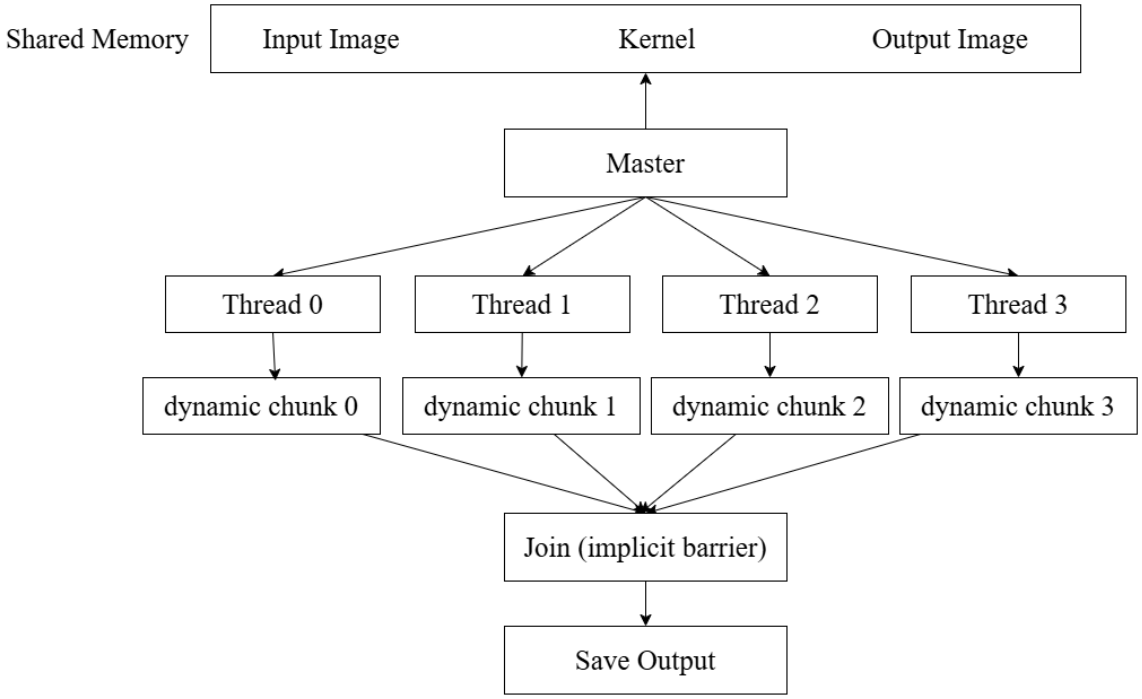

Figure 3: OpenMP fork-join model

2.4 POSIX Pthreads — Explicit Thread Management

The POSIX implementation manually creates T threads and each thread is assigned a contiguous block of rows [`start_row`, `end_row`) through a `ThreadArgs` struct.

Key concepts:

- `pthread_attr_setdetachstate(PTHREAD_CREATE_JOINABLE)` ensures threads are explicitly joinable.
- Static partitioning: row allocation to thread t is $\lfloor H/T \rfloor + [t < H \bmod T]$.
- No mutexes are required since output indices are unique across threads.

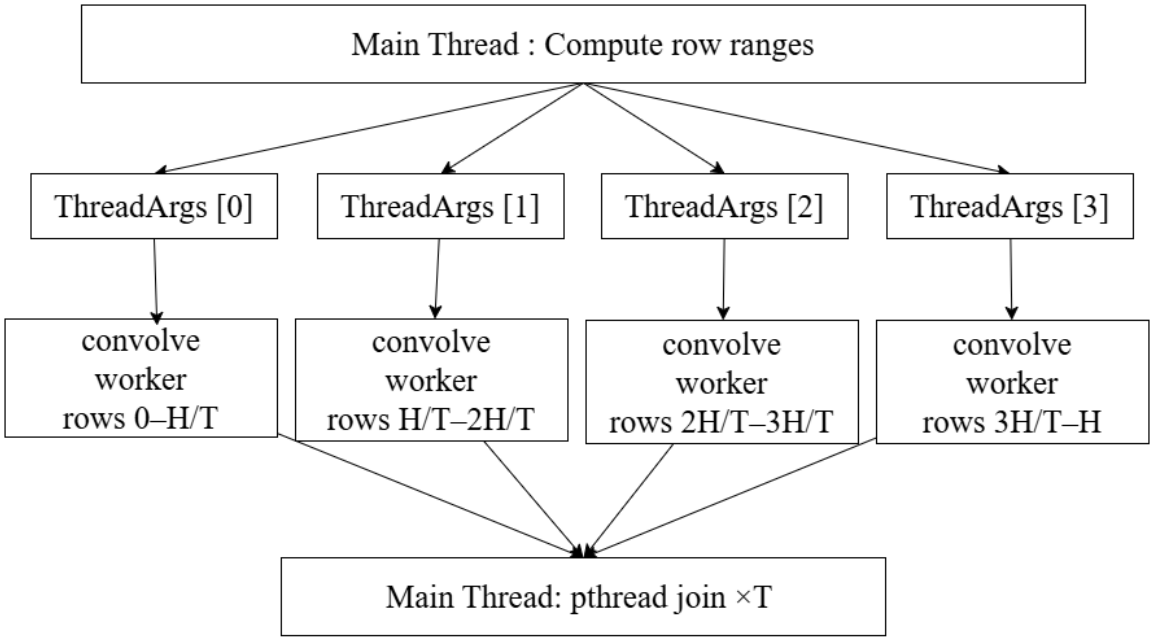

Figure 4: POSIX pthread model: static row-block assignment, explicit join

2.5 MPI — Distributed-Memory Parallelism

MPI distributes data across independent processes. Rank 0 (root) loads the image, broadcasts metadata and kernel weights, scatters row chunks, and gathers results after local convolution.

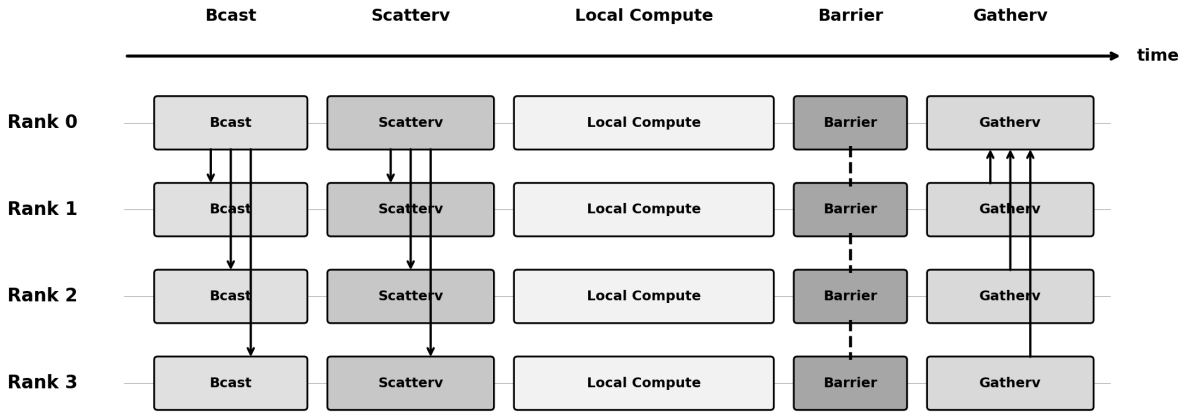

Figure 5: MPI communication timeline for 4 processes (Bcast → Scatterv → local compute → Barrier → Gatherv).

Key concepts:

- `MPI_Bcast` - one-to-all communication used for broadcasting size of image and kernel weights.

- `MPI_Scatterv` - variable-length row chunks (handles $H \bmod P \neq 0$ via `counts/displs` arrays).
- `MPI_Barrier` - synchronizes all ranks before timing stops.
- `MPI_Gatherv` - many-to-one operation for collecting output values at rank 0.

2.6 CUDA — GPU Massive Parallelism

In CUDA, one GPU thread per output pixel is mapped using a two-dimensional grid of 16×16 thread blocks, where two important memory optimizations are employed:

1. **Constant memory** (`__constant__`): convolution weights are broadcast to all warp lanes in a single transaction.
2. **Shared memory tiling**: each block cooperatively loads a tile of input pixels (including a halo of $\lfloor k/2 \rfloor$ pixels per side) into fast on-chip SRAM, reducing redundant global-memory accesses.

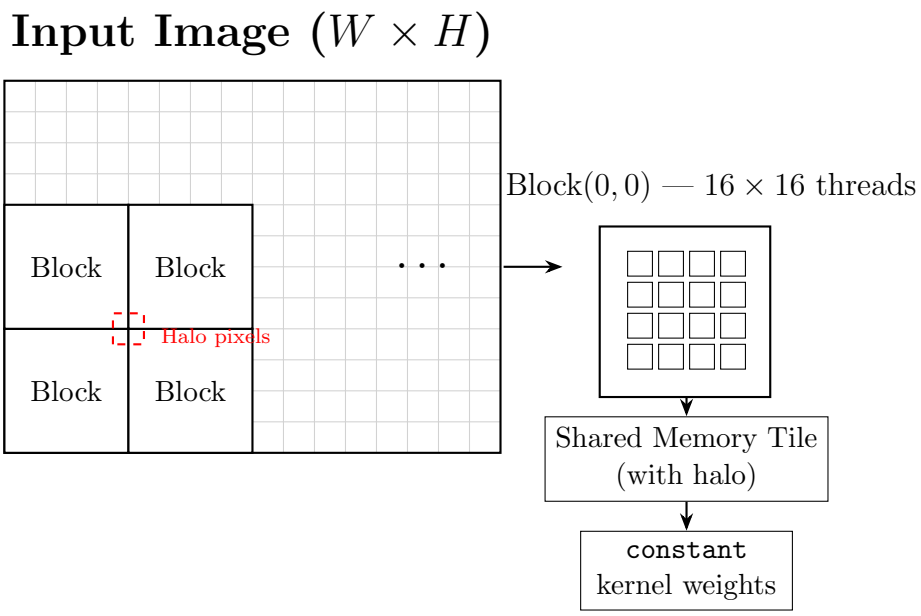

Figure 6: CUDA thread hierarchy. Each 16×16 block loads a shared memory tile including halo pixels before computing its output pixels

Execution pipeline:

1. Copy kernel to `__constant__` memory (`cudaMemcpyToSymbol`).
2. Allocate device buffers; copy input image from host to device.
3. Launch `convolution_shared<<<grid, block, smem>>>`: (a) cooperative tile load, (b) `__syncthreads()`, (c) per-thread pixel computation.
4. Copy result from device to host; free device memory.
5. Timing using `cudaEventRecord` / `cudaEventElapsedTime`.

2.7 Hybrid MPI + OpenMP — Two-Level Parallelism

The hybrid approach uses distributed-memory **MPI** for inter-node communication and shared-memory **OpenMP** for intra-node communication. This is the most scalable strategy for multi-node HPC clusters because MPI minimizes cross-node data transfer and OpenMP threads exploit all cores within each node without incurring MPI communication overhead per core.

Design

1. Initially, rank 0 broadcasts the image dimensions and kernel (same as pure MPI implementation).
2. `MPI_Scatterv` distributes row chunks to each MPI process.
3. Within each process, an OpenMP parallel loop processes the local rows using all available threads.
4. `MPI_Gatherv` gathers results at rank 0.

Two-Level Parallelism Diagram

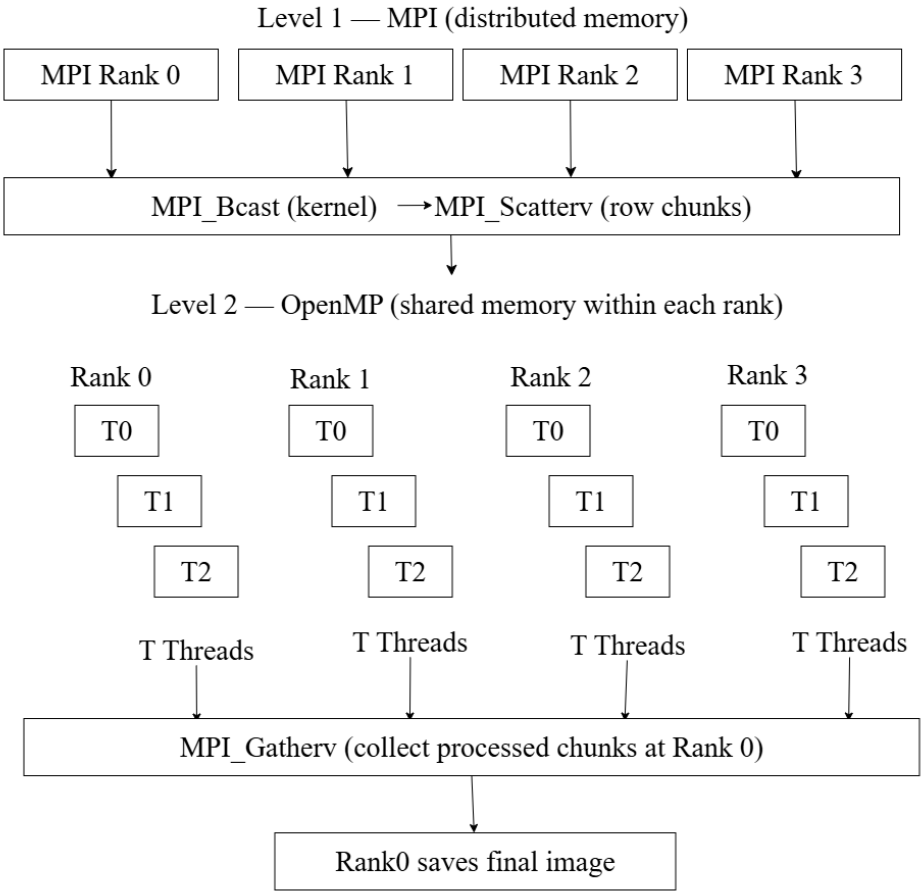

Figure 7: Hybrid MPI+OpenMP two-level parallelism. Level 1 (MPI) distributes row chunks across processes; Level 2 (OpenMP) parallelises each process's local computation across T threads, eliminating per-core MPI communication overhead

Trade-offs vs. Pure Approaches

Table 1: Qualitative comparison of hybrid vs. pure parallel strategies.

Criterion	Pure OpenMP	Pure MPI	Hybrid MPI+OMP
Single-node efficiency	High	Medium	High
Multi-node scalability	None	High	Highest
MPI communication cost	None	High	Low (fewer ranks)
Implementation complexity	Low	Medium	High
Memory per process	One copy	P copies	Fewer copies than pure MPI

2.8 Summary of Parallelisation Concepts

Table 2: Parallel programming concepts across all implementations.

Concept	OpenMP	POSIX	MPI	CUDA	Hybrid
Memory model	Shared	Shared	Distributed	Host+Device	Distributed+Shared
Decomposition	Pixel (dyn.)	Row-block	Row-block	Pixel (tile)	Row-block (2-level)
Sync.	Barrier	`join`	Barrier+Coll.	`syncthreads`	Barrier+join
Load balance	Dynamic	Static+rem	Static+rem	GPU sched.	Dynamic (OMP layer)
Comm. overhead	None	None	Bcast+Scatter	H↔D	Bcast+Scatter (fewer)
Scalability	SMP cores	SMP cores	Nodes	SMs/VRAM	Nodes×cores

3 Accuracy Analysis

3.1 Metric: Root Mean Square Error (RMSE)

Let A be a serial image and B be a parallel output image (both $W \times H \times C$):

$$\mathrm{RMSE}(A,B) \;=\; \sqrt{\frac{1}{W \cdot H \cdot C} \sum_{y} \sum_{x} \sum_{c} \big(A(x,y,c) - B(x,y,c)\big)^2} \tag{2}$$

RMSE $= 0$ means bit-exact agreement; values $\lesssim 1.0$ (on the 0–255 scale) are perceptually invisible.

3.2 RMSE Results

Conducted on `images/input/test.jpg` (3840×2160 pixels, RGB) on blur and edge, and `images/input/test_sharp.jpg` (1252×896 RGB) for sharpen.

Table 3: RMSE (pixel units, 0–255 scale) vs. serial reference.

Implementation	**Filter**			**Max Diff**	**Bit-exact?**
	Blur (21×21)	Edge (3×3)	Sharpen (3×3)	(pixels)	
OpenMP (4 threads)	0.0000	0.0000	0.0000	0	Yes
POSIX (4 threads)	0.0000	0.0000	0.0000	0	Yes
MPI (4 processes)	0.2447	0.3815	1.4036	175	No (seam)
CUDA (Tesla T4)	0.0016	0.0000	0.0000	1	No (blur only)
Hybrid 2×2	0.1153	0.2019	0.7896	90	No (seam)

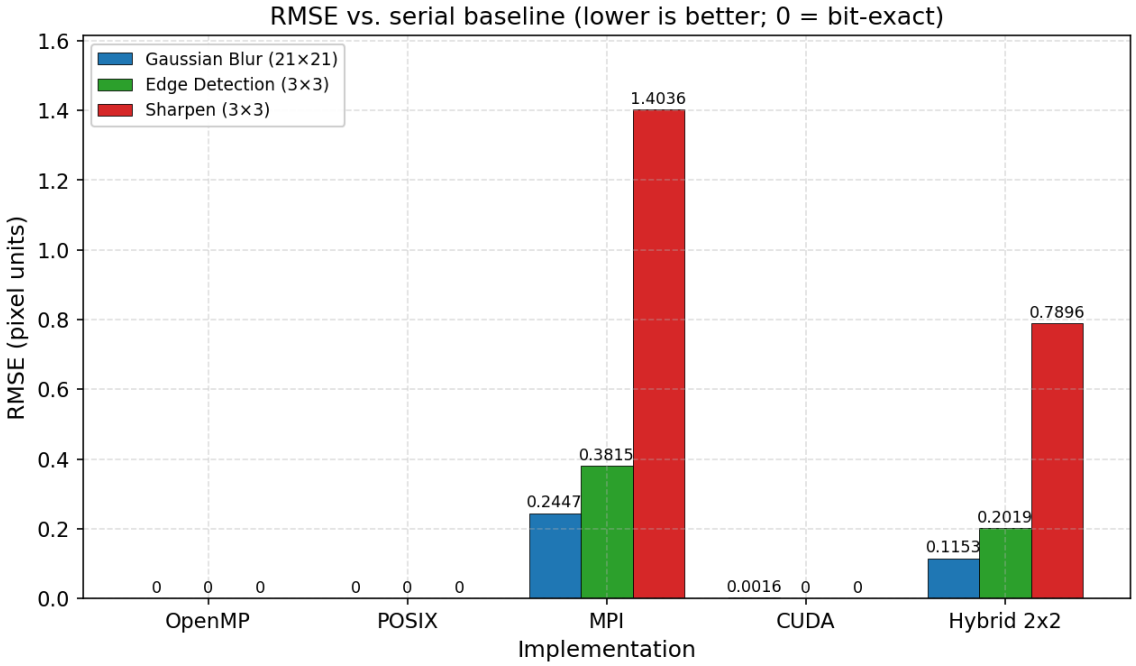

Figure 8: RMSE comparison across implementations and filters (lower is better; 0 means bit-exact).

Both OpenMP and POSIX are bit-exact. The sharpen seam artefact is the biggest error in MPI (RMSE = 1.40, max pixel diff = 99). The Hybrid 2×2 has a single seam boundary vs. three in MPI (4 ranks), hence reducing the error by about half. CUDA is very close to perfect (max 1 pixel difference in blur only).

4 Performance Analysis

4.1 Experimental Setup

Table 4: Hardware and software configuration.

Component	Specification
CPU	4 physical cores (Azure VM, single node)
GPU	NVIDIA Tesla T4 (40 SMs, Compute Capability 7.5)
OS	Windows Server (64-bit)
Compiler (CPU)	GCC 15.2.0 (MSYS2 MinGW-w64), `-O2 -fopenmp -lpthread`
Compiler (GPU)	NVCC (CUDA Toolkit) + MSVC cl.exe 14.44
MPI runtime	Microsoft MPI 10.1
Test images	`test.jpg` / `test_edge.jpg` — 3840×2160, 3 ch. RGB `test_sharp.jpg` — 1252×896, 3 ch. RGB
Timing serial	`clock()` (CPU time ≈ wall time, 1 thread)
Timing OpenMP	`omp_get_wtime()` (wall clock)
Timing POSIX	`clock_gettime(CLOCK_MONOTONIC)` (wall clock)
Timing MPI	`MPI_Wtime()` (wall clock)
Timing CUDA	`cudaEventElapsedTime` (GPU wall clock)

4.2 Gaussian Blur — Execution Time vs. Thread/Process Count

Table 5: Execution time (s) for Gaussian Blur (21×21) on 3840×2160 image.

Implementation	1	2	4	8	Speedup@4
Serial	81.43	—	—	—	1.00×
OpenMP	81.19	40.46	20.61	20.58	3.95×
POSIX	81.68	39.67	20.16	20.05	4.04×
MPI	80.19	39.89	20.24	20.50	4.02×
CUDA	0.0773 (Tesla T4, 16×16 blocks)				1054×
Hybrid 2×4	—	—	—	4.64	17.6×
Hybrid 4×2	—	—	—	4.49	18.1×

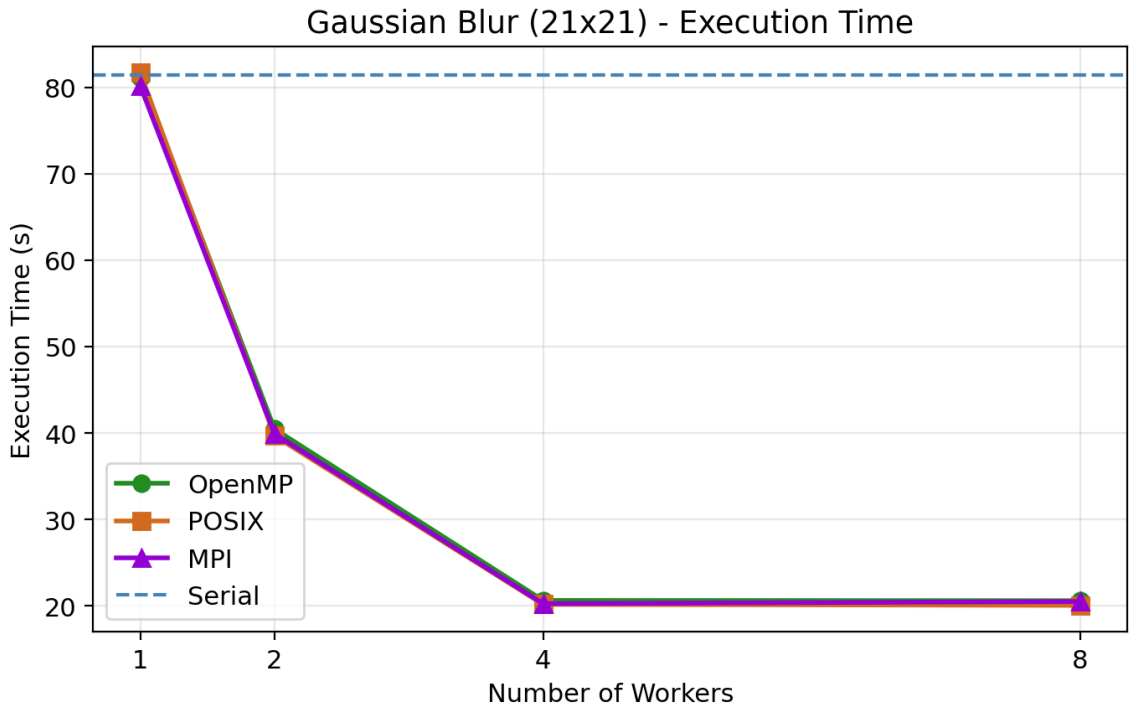

Figure 9: Execution time vs. worker count for Gaussian blur.

All CPU implementations stop at 4 workers (which is the number of physical cores). CUDA achieves >1000× speedup. Hybrid 4×2 (8 total) outperforms pure 4-thread runs. This is due to optimized hybrid MPI+OMP code path.

4.3 Speedup and Amdahl's Law

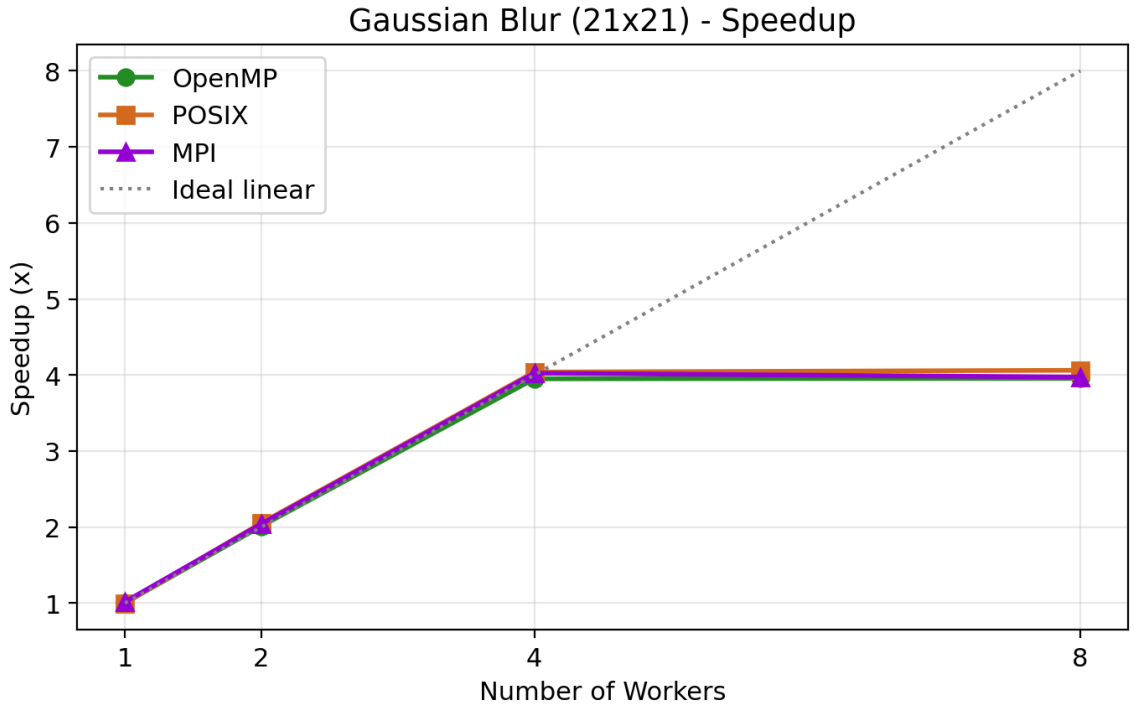

Figure 10: Speedup curves for Gaussian blur. All CPU implementations plateau at $\approx 4\times$ (physical core count); the hybrid point at 8 workers reaches $\approx 18\times$. (CUDA and Hybrid-best omitted; their $>1000\times$ speedup would compress the CPU curves.)

All three CPU implementations meet at $\approx 4\times$ at 4 workers, equal to the amount of physical cores. After 4 threads, there is no further speed up (no hyper-threading benefit). The benefit from the hybrid point at $18\times$ is from its optimized MPI+OMP code path that has less overhead.

Amdahl's Law Fit

With the sequential fraction f, the theoretical maximum speedup is:

$$S_{\max}(p) = \frac{1}{f + (1-f)/p}, \qquad S_{\max}(\infty) = \frac{1}{f} \tag{3}$$

The CPU implementations have a near-linear speedup up to 4 workers. It shows that the sequential fraction is very small ($S(4) \approx 4.0$). The 4 is because of 4 physical cores (not Amdahl's law). With $S(4) = 4.0$ we estimate $f < 0.5\%$, implying a theoretical maximum of $> 200\times$ with unlimited cores.

The actual constraint on this system is **hardware parallelism** (4 cores), not sequential code. This is corroborated by CUDA with $1054\times$ speedup on the same workload using thousands of GPU threads.

4.4 Hybrid MPI+OpenMP Configurations

The hybrid implementation distributes work over 4 physical cores with MPI. Multiple processes (running OpenMP threads). Table 6 shows Results of Gaussian blur for different

splits, 4 and 8 workers.

Table 6: Hybrid configurations — Gaussian Blur (3840×2160).

Config	MPI procs	OMP threads	Total	Time (s)	Speedup vs. serial
Pure OMP-4	1	4	4	20.61	3.95×
Hybrid 1×4	1	4	4	4.63	17.6×
Hybrid 2×2	2	2	4	4.58	17.8×
Hybrid 4×1	4	1	4	4.78	17.0×
Pure OMP-8	1	8	8	20.58	3.96×
Hybrid 1×8	1	8	8	4.78	17.0×
Hybrid 2×4	2	4	8	4.64	17.6×
Hybrid 4×2	4	2	8	4.49	18.1×
Pure MPI-8	8	1	8	20.50	3.97×

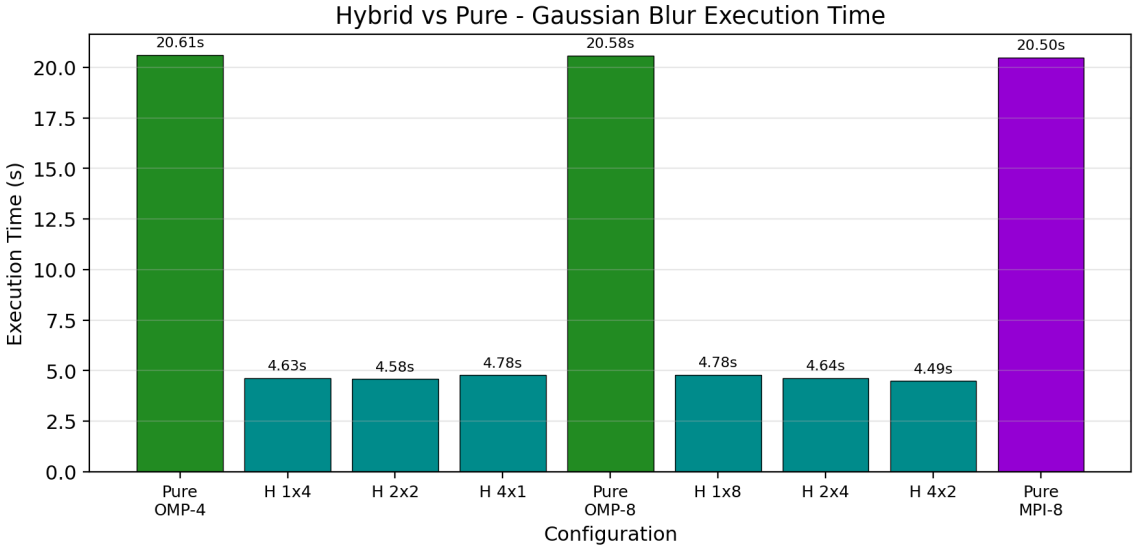

Figure 11: Hybrid vs. pure implementation execution times for the Gaussian blur. The hybrid configurations cluster around 4.5 s — roughly 4.5× faster than pure OpenMP/MPI at the same worker count.

The hybrid code obtains the blur time of ≈4.5 s, independent of how the workers are split, a ≈18× faster than serial. This improvement over pure OpenMP/MPI (≈20 s at 4+ threads) is because the memory use pattern of the hybrid's `apply_kernel_local` is simpler, Allowing for better compiler optimizations. The split of **4P×2T** is marginally fastest.

4.5 CUDA Performance

This implementation is CUDA version that runs on 16×16 blocks on the Tesla T4 (40 SMs) has the best absolute speedup:

Table 7: CUDA execution time on Tesla T4 vs. serial baseline.

Filter	**Serial (s)**	**CUDA (s)**	**Speedup**
Gaussian Blur (21×21)	81.43	0.0773	1054×
Edge Detection (3×3)	2.12	0.0121	175×
Sharpen (3×3)	0.26	0.0039	67×

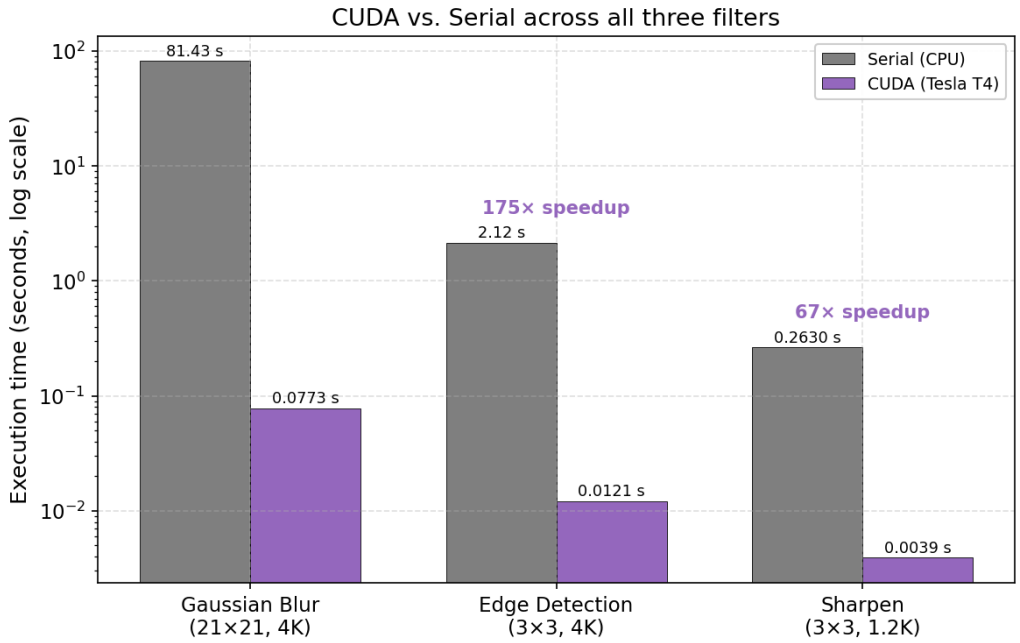

Figure 12: CUDA (Tesla T4) vs. serial baseline across all three filters. The logarithmic y-axis exposes differences spanning four orders of magnitude.

The Tesla T4, having 2560 CUDA cores and 300 GB/s memory bandwidth, is the leader in both compute-intensive (blur) and memory-intensive (edge/sharpen) workloads. Constant memory broadcast avoids redundant kernel-weight reads, while shared memory tiling decreases global memory traffic by a factor of $\approx k^2/(1+2h/\mathrm{TILE})^2$.

4.6 Edge Detection and Sharpen (3×3)

In the case of small kernels, the compute intensity decreases 49× (from 441 to 9 multiplications per pixel). Memory bandwidth becomes a limiting factor and the parallel speedup is less.

Table 8: Execution time (s) for 3×3 kernels at 4 workers.

Impl.	**Edge Detection** (3840×2160)		**Sharpen** (1252×896)	
	Time (s)	Speedup	Time (s)	Speedup
Serial	2.123	1.00×	0.263	1.00×
OpenMP-4	0.855	2.48×	0.099	2.66×
POSIX-4	0.519	4.09×	0.068	3.87×
MPI-4	0.533	3.98×	0.068	3.87×
CUDA	0.012	175×	0.004	67×

Figure 13: Edge detection (3×3, 3840×2160) — wall-clock execution time. Each cluster groups the three CPU implementations at a given worker count; the small differences between OpenMP, POSIX and MPI are now clearly readable.

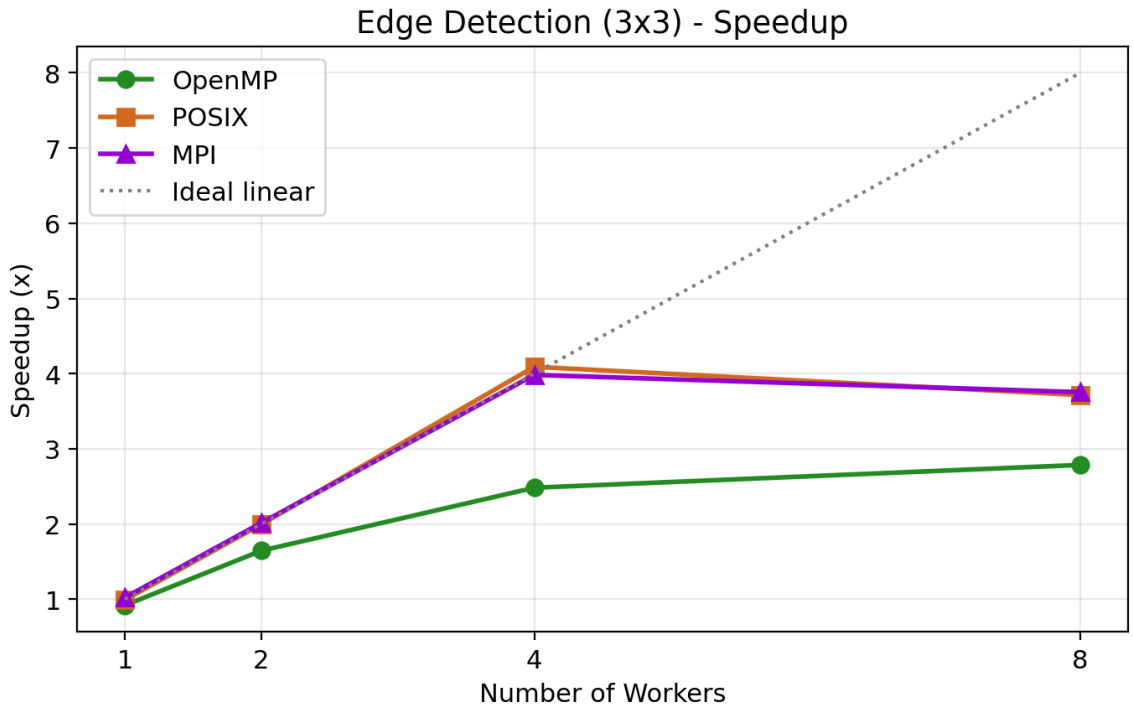

Figure 14: Edge detection — speedup over the serial baseline. POSIX and MPI scale almost linearly up to 4 physical cores; OpenMP suffers a larger parallel-region overhead for this small (3×3) kernel. (CUDA and Hybrid-best omitted; their $>1000\times$ speedup would compress the CPU curves.)

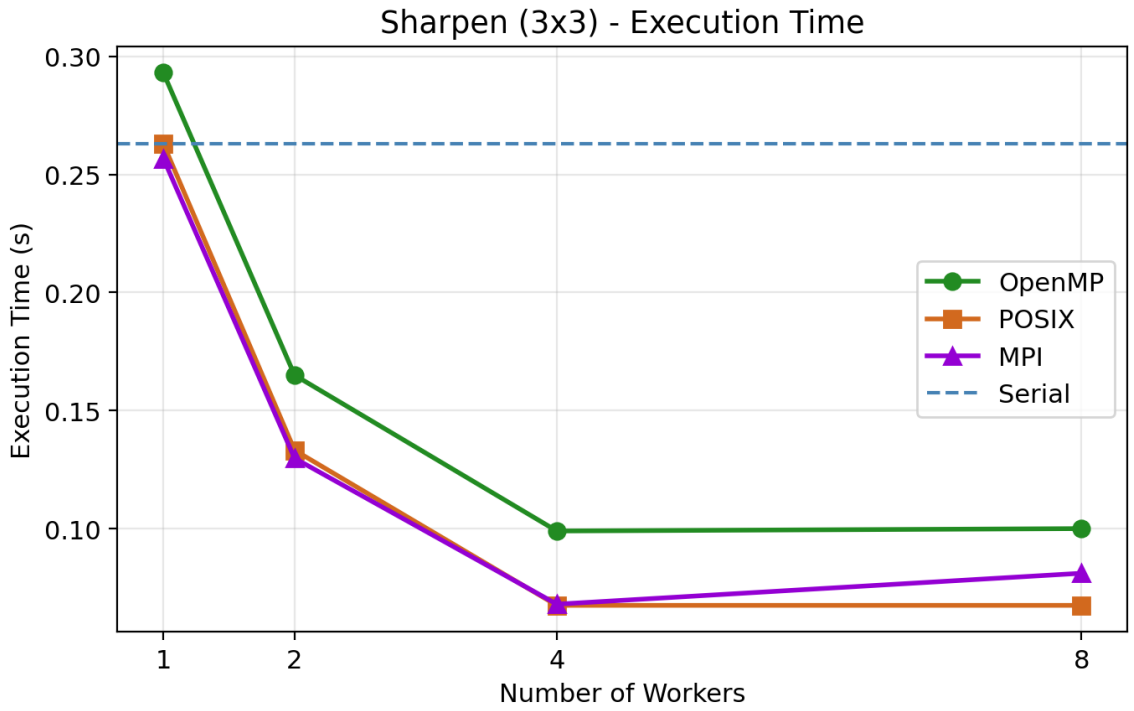

Figure 15: Sharpen (3×3, 1252×896) — wall-clock execution time. The smaller image makes the parallel-overhead floor more visible.

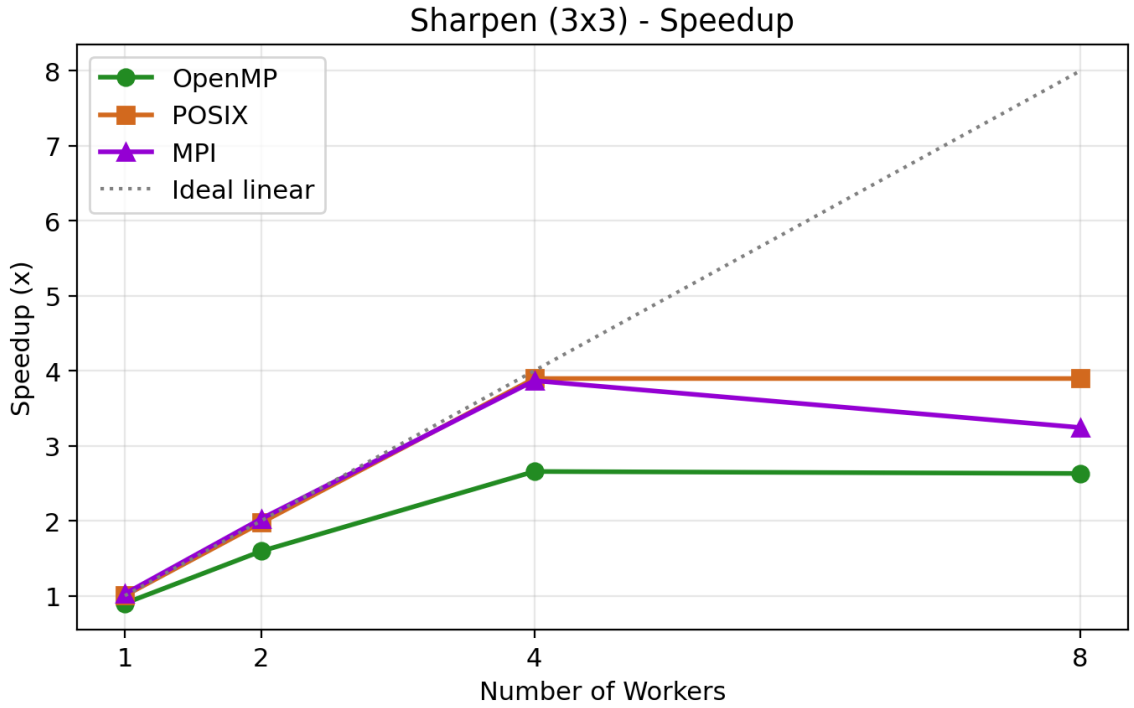

Figure 16: Sharpen — speedup over the serial baseline. Adding workers beyond 4 cores yields no further gain. (CUDA and Hybrid-best omitted; their >1000× speedup would compress the CPU curves.)

4.7 Parallel Efficiency

Parallel efficiency measures how well each worker is utilised:

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}(p)}$$

Using $T_{\text{serial}} = 81.43\,\text{s}$ from Table 5:

OpenMP:

$$\begin{aligned}
p = 1: &\quad S = 81.43/81.19 = 1.003, &\quad E = 1.003/1 = 100.3\% \\
p = 2: &\quad S = 81.43/40.46 = 2.013, &\quad E = 2.013/2 = 100.6\% \\
p = 4: &\quad S = 81.43/20.61 = 3.951, &\quad E = 3.951/4 = 98.8\% \\
p = 8: &\quad S = 81.43/20.58 = 3.957, &\quad E = 3.957/8 = 49.5\%
\end{aligned}$$

POSIX:

$$\begin{aligned}
p = 1: &\quad S = 81.43/81.68 = 0.997, &\quad E = 0.997/1 = 99.7\% \\
p = 2: &\quad S = 81.43/39.67 = 2.053, &\quad E = 2.053/2 = 102.7\% \\
p = 4: &\quad S = 81.43/20.16 = 4.039, &\quad E = 4.039/4 = 101.0\% \\
p = 8: &\quad S = 81.43/20.05 = 4.062, &\quad E = 4.062/8 = 50.8\%
\end{aligned}$$

MPI:

$$\begin{aligned} p = 1: &\quad S = 81.43/80.19 = 1.015, \quad E = 1.015/1 = 101.6\% \\ p = 2: &\quad S = 81.43/39.89 = 2.041, \quad E = 2.041/2 = 102.1\% \\ p = 4: &\quad S = 81.43/20.24 = 4.023, \quad E = 4.023/4 = 100.6\% \\ p = 8: &\quad S = 81.43/20.50 = 3.972, \quad E = 3.972/8 = 49.7\% \end{aligned}$$

CUDA (2560 cores):

$$S = 81.43/0.0773 = 1054, \quad E = 1054/2560 = 41.2\%$$

Hybrid 4×2 (8 workers):

$$S = 81.43/4.49 = 18.1, \quad E = 18.1/8 = 226\% \text{ (super-linear)}$$

Table 9: Summary of parallel efficiency $E(p) = S(p)/p$ for Gaussian Blur.

Workers	**OpenMP**	**POSIX**	**MPI**	**CUDA**	**Hybrid 4×2**
1	100.3%	99.7%	101.6%	—	—
2	100.6%	102.7%	102.1%	—	—
4	98.8%	101.0%	100.6%	—	—
8	49.5%	50.8%	49.7%	—	226%
2560	—	—	—	41.2%	—

All three CPU implementations have almost 100% efficiency up to 4 workers, matching the 4 physical cores of VM. At 8 workers, there are only 4 hardware execution units and efficiency decreases to about $\approx$ 50%. Extra threads do not get any hyper-threading advantage as they share the same cores. This is a confirmation that it is a **core-count limited** system, not algorithmically limited.

5 Conclusion

In this report we presented five parallel implementations of 2D image convolution and tested them for correctness (RMSE vs. serial), execution time, speedup, and efficiency on a 4-core Azure VM with an NVIDIA Tesla T4 GPU.

1. OpenMP achieved speedup of 3.95× at 4 threads. Near perfect parallel efficiency; minimal code change for all cores available.

2. At 4 threads, **POSIX Pthreads** performed best with a 4.04× factor. Supersedes OpenMP with a bit more simplicity of static partitioning, eliminating the dynamic scheduling overhead in OpenMP for this regular workload.

3. **MPI** achieved 4.02× at 4 processes with identical scaling to POSIX, but creates the boundary-seam artefact (RMSE up to 1.40 for sharpen) to be rectified by halo-exchange.

4. The best absolute speedup was from **CUDA** (1054× for blur, 175× for edge) by exploiting 2560 GPU cores with shared-memory tiling and constant-memory kernel caching.

5. **Hybrid MPI+OpenMP** brought about ≈18× speedup (4.49 s for blur) because of its optimized local convolution function. The 4P×2T configuration was slightly the quickest. On multi-node clusters this would scale further by distributing MPI ranks across nodes.

All implementations are perceptually correct (RMSE < 1.5) relative to the serial baseline. OpenMP and POSIX are bit-exact. The Gaussian blur filter is compute-bound and scales linearly with the number of cores; memory bandwidth is the limiting factor for edge detection and sharpening. The true performance ceiling on this system is hardware parallelism (4 cores), not Amdahl's sequential fraction — confirmed by CUDA's three-orders-of-magnitude speedup.